Another line of work that can be used to test multi-threaded programs that take inputs from unbounded domains are approaches based on *generalized symbolic execution*, starting with the work of Khurshid et al. [2003], which took advantage of the partial order and symmetry reduction capabilities of the model checker component of the Java PathFinder tool. Broadly speaking, most early approaches based on symbolic execution do not achieve the reduction in explored traces that DPOR techniques can offer and do not define any notion of optimality.

Recently, Huang [2015] characterized a generalization (i.e., weaker form) of Mazurkiewicz traces based on a new criterion: the *maximal causal model* for a concurrent computation from a given execution trace, a notion defined by Serbanuta et al. [2012]. Maximal causality characterizes the largest possible set of equivalent interleavings in each (weaker form of) Mazurkiewicz trace by taking semantic information into consideration, namely the values of reads and writes. When applied in stateless model checking, maximal causality enables exploration of the entire state space of a concurrent program with respect to a given input with a provably minimal number of executions. The corresponding algorithm, called Maximal Causality Reduction (MCR), relies on an offline constraint analyzer to formulate constraints that are then solved using an SMT solver. Its implementation, which is not publicly available, was compared against an implementation of the original DPOR algorithm with iterative context bounding [Musuvathi and Qadeer 2007] and was found to outperform it in most cases. In view of this result, it would be interesting to compare MCR, both theoretically and experimentally, with a better DPOR algorithm, such as optimal-DPOR, at some point.

13 CONCLUDING REMARKS

We have presented a new theoretical foundation for partial order reduction, based on *source sets*, and two new algorithms for dynamic partial order reduction, called *source-DPOR* and *optimal-DPOR*. The latter algorithm, which combines source sets with a novel mechanism called *wakeup trees* to achieve optimality, is the first DPOR algorithm to be provably optimal in the sense that it is guaranteed both to completely explore the minimal number of executions and avoid even initiating executions that lead to sleep set blocking. As shown in the experimental evaluation, the extra overhead of maintaining wakeup trees is very moderate in practice (never more than 10% in our experiments), which is a good trade-off for having an optimality guarantee and the possibility to run arbitrarily (exponentially) faster than other DPOR algorithms. However, as also shown in this article, wakeup trees can be exponential in size in situations where an exponential number of executions needs to be explored by the optimal algorithm. In such situations, the source-DPOR algorithm, which maintains less information than the optimal algorithm, could be used as a fallback in a tool that implements both algorithms. Another reason to prefer the source-DPOR algorithm is its implementation simplicity; indeed, one only needs to substitute persistent sets with source sets in an implementation of the original DPOR algorithm of Flanagan and Godefroid [2005b] to obtain an implementation of source-DPOR.

Due to its implementation simplicity, we have chosen the source-DPOR algorithm to be the core algorithm used in Nidhugg, a stateless model checking tool for C/pthreads programs under sequential consistency and also under relaxed memory models, and achieved performance that outperforms state-of-the-art tools of similar capabilities [Abdulla et al. 2015]. Recently, we have applied Nidhugg to systematically test various flavors of the Read-Copy-Update (RCU) synchronization mechanism of the Linux kernel. We have been able to reproduce, within seconds, various safety and liveness bugs that have been reported for Tree RCU's implementation, and have verified the Grace-Period guarantee, the basic guarantee that RCU offers, on non-preemptible builds of several Linux kernel versions. The source-DPOR algorithm has been instrumental in verifying this